

Advancements in AI-Driven Customer Service

^{1st} Mona Esmaili
*dept of Electrical &
Computer Engineering
University of New Mexico
Albuquerque, USA
mesmaeili@unm.edu*

^{2nd} Mohammad Ahmadi
*dept of Mathematics & Statistics
University of New Mexico
Albuquerque, USA
mahmadi@unm.edu*

^{3rd} Mohammad David Ismaeil
*dept of Mechanical Engineering
Islamic Azad University
Tehran, Iran
ismaelmdavid@gmail.com*

^{4th} Sharareh Mirzaei
*dept of Civil, Construction,
and Environmental Engineering
University of New Mexico
Albuquerque, USA
shmirzaei@unm.edu*

^{5th} Jorge Canales Verdial
*dept of Math, Science & Engineering
Central New Mexico Community College
Albuquerque, USA
jcanalesverdia@cnm.edu*

Abstract—Chatbots have gained more attention for resolving straightforward queries by retrieving data from relevant knowledge bases. As the capabilities of these interactive platforms have advanced, they have been integrated into user behavior, with customers using them for insights into past purchases or more information on future acquisitions. In this context, this paper presents the Amazon Question Answering Agent. This tool is designed to answer users' frequently asked questions regarding products in Amazon's electronics section. This objective will be achieved by embedding users' queries and a pre-existing dataset, derived from users' past questions, into a unified embedding space and then ranking the results to ensure the retrieval of relevant answers for optimal user satisfaction. The research evaluates four major algorithms—BM25, Word2Vec, GloVe, and FastText—for their effectiveness in generating sentence embeddings. The results showed that BM25 method outperforms the others in terms of accuracy and time efficiency, which increases the usefulness of the suggested Amazon Question Answering Agent. The study contributes to advancing chatbot capabilities in efficiently addressing user inquiries within e-commerce, promising enhanced engagement and satisfaction.

Index Terms—Information Retrieval (IR), Natural Language Understanding (NLU), Question Answering (QA), BM25, Word2Vec, GloVe, FastText, Ranking, Neural Embeddings

I. INTRODUCTION

The chatbot is a software tool powered by artificial intelligence, developed to maintain conversation and find relevant responses to address users' inquiries. Chatbots can be used across a variety of domains, such as managing personal bank accounts, checking the weather, and shopping. As the popularity of digital assistants grows, it becomes increasingly common for users to utilize these tools to gain information about products [1]. Consequently, in this paper, we have created an Amazon QA chatbot capable of answering users' queries by retrieving the most pertinent response from a pool of available answers.

Delving more into detail, the input to our system is a natural language question about a product, and the output is

the top-ranked answer selected from a large dataset of potential answers. To enhance manageability, our research specifically targets the electronics category, which encompasses more than 300,000 question-and-answer pairs, in addition to other meta-data. For this purpose, we have developed multiple algorithms and models to optimize and improve the accuracy of the chatbot. Furthermore, we plan to develop a web demo that will enhance the direct front-end interface of our agent [2].

II. BACKGROUND

A. Motivation

Digital assistance is witnessing significant expansion. Its usage ranges from managing smart homes to teaching social skills to children. Nonetheless, there is a noticeable gap in providing solutions for answering product-related inquiries due to various technical and theoretical hindrances.

Challenges in natural language understanding (NLU) involve developing insights into a specific domain and its unique characteristics, as well as handling noisy inputs from users' speech and accents [3]. Additionally, effective information retrieval (IR) requires writing extensive rule sets or translating the user's natural language and intent into a format that can be comprehended computationally [3]. In both approaches, it is difficult to deliver accurate answers. On the psychological front, users may not always be clear or may struggle to articulate their inquiries. These reasons make the question-answering system both challenging and interesting at the same time [4].

B. Related Work

Considering the significant impact of conversational agents capable of providing relevant responses to questions, the development of these chatbots has been a captivating and dynamic area of research for many years [5]. By reviewing some previous papers related to this field, specifically focusing

on the use of information retrieval for answering questions, we aim to highlight the importance of this area.

The study *An Information Retrieval Approach to Short Text Conversation* [6] addresses the fundamental challenges of conversational systems, focusing on ensuring that the computer provides reasonable responses to human input through information retrieval. For this purpose, they formalize the input as a search problem and then apply an information retrieval strategy, specifically a RankingSVM, to provide answers. The retrieval-based model functions more intelligently when applied to social media data [7]. Moreover, *Chatbot engines using unstructured documents* [8] introduced the DocChat method within the chatbot engine to explore unstructured information to answer human questions, demonstrating the efficacy of the retrieval-based method in finding the most relevant question and answer pairs directly [8]. On the other hand, generation-based methods focus on creating new answers. Previous approaches, whether search or generation-focused, depend heavily on extensive dialogue sentence pairs for training data. In specific fields, obtaining a large dialogue corpus is more difficult, while ordinary text is more readily available. This paper proposes an efficient method for generating informative queries [9].

Transitioning towards statistical methodologies, the study *"Data-Driven Response Generation in Social Media"* utilizes Statistical Machine Translation (SMT) to generate responses [10]. This approach resolves the challenges associated with the broad spectrum of possible answers and the significant portion of unaligned words and phrases. The assessment, conducted using BLEU scores, demonstrates that SMT outperforms conventional Information Retrieval (IR) methods. Remarkably, SMT results are preferred over human responses in 15% of the cases [11] [12].

In recent research, more neural-based methods are being studied. The "Neural Responding Machine for Short-Text Conversation" introduces a neural network-based response generator for short-text interactions, known as the Neural Responding Machine (NRM) [13]. This method employs an encoder-decoder framework, formalizing the generation of responses as a decoding task. Both the encoding and decoding processes utilize Recurrent Neural Networks (RNNs), specifically Gated Recurrent Units (GRUs). The NRM models are capable of generating grammatically and contextually correct responses in more than 75% of the inputs [14]. Additionally, *A Neural Conversational Model* introduces a seq2seq-based language model that can be used to train a conversational engine [15]. This approach involves a recurrent neural network that processes the input sequence one token at a time and similarly predicts the output sequence token by token. During training, the actual output sequence is provided, enabling learning via backpropagation. The model's goal is to optimize the cross-entropy of the correct sequence given its context [2]. In inference, since the actual output sequence cannot be observed by users, the model uses the predicted output token to forecast the subsequent output, thus enhancing its predictive accuracy over time [16] [4].

III. PROBLEM DESCRIPTION & DATASETS

The objective of this paper is to respond to consumer queries in text format, typically consisting of a sentence or two, utilizing an Amazon knowledge-based chatbot. To accomplish this, we initially embed both the query and all potential questions into a common vector space. Subsequently, we identify the question that most closely aligns with the query and present the answer associated with that question. In parallel, we explore the Direct-to-Answer approach. This method involves encoding the consumer's query and directly matching it with available answers to select the most appropriate one. Although this approach is more straightforward, it may require specialized machine reasoning.

A. Datasets

Our dataset originates from the Amazon Q&A collections created by the McAuley team at UCSD and is limited to the Electronics category. It comprises 314,263 instances. The format of the dataset is outlined below:

```
{
  "asin": "B000050B6Z",
  "questionType": "yes/no",
  "answerType": "y",
  "answerTime": "Aug 8, 2014",
  "unixTime": 1407481200,
  "question": "Can you use this unit with GEL shaving cans?",
  "answer": "Yes. If the can fits in the machine it will dispense hot gel lather. I've been using my machine for both , gel and traditional lather for over 10 years."
}
```

Fig. 1. Example of a dataset format.

B. Evaluation Methods

Upon reviewing the data, we noted that many questions lack clarity because a single inquiry can be interpreted in multiple ways and still yield the same response. Therefore, to enhance the quality of the user experience, our chatbot needs to process a large variety of queries, even if they are not worded as in our dataset. It is unlikely that users will phrase a question exactly as it appears in our dataset. To address this problem, we must manually annotate subsets of the data and revise questions.

To elucidate this issue further, let us consider an example. Imagine a user poses the question, "How is the warranty for this monitor?" If our dataset contains this exact question, then matching algorithms will easily identify the precise answer. However, if this query is worded differently in the chatbot's dataset, despite the question's concept being the same, finding the answer becomes less straightforward.

It is more likely that the input queries will not exactly match those in our dataset; hence, our evaluation must account for these differences. Our algorithms are designed to handle such deviations. However, without a collection of varied modified queries, assessing the performance of the algorithms is difficult. Although developing this project is time-consuming, we have devoted significant effort to creating such a dependable test set.

As part of this result, we have developed a set of 1200 manually edited queries that serve as paraphrases of the original inputs. For each example query, there is an accompanying set of 100 questions for evaluation, consisting of 99 unrelated samples and one original question corresponding to the query. Ideally, our algorithm should be able to match the original question with this set of 100 using the revised query to find the best answer. In the Direct-to-Answer method, approximately 45,000 answers are available for evaluation, having been selected from the entire dataset based on specific criteria. More specifically, we filter examples that exhibit any of the following characteristics:

- (a) The type of question is Yes/No.
- (b) Answers contain fewer than 5 tokens.
- (c) Answers exceed 50 tokens in length.

Lastly, in the Raw Answers approach, the comprehensive set of over 300,000 answers forms the evaluation candidate set.

C. Metrics

C. Evaluation Metrics

We assess our models' effectiveness by calculating their overall accuracy. Specifically, each case provides the algorithm with one opportunity to accurately identify a match, a process repeated 1200 times. Therefore, the accuracy rate is determined by the percentage of instances in which the algorithm efficiently selects the correct match, serving as a direct measure of the algorithm's capability. Additionally, we perform manual evaluations of the responses for a qualitative analysis of their performance.

IV. EXPERIMENTS

A. Solution Architecture

Fig. 2 shows our design for the solution architecture of the chatbot. Once a user utterance comes in, it is tokenized and parsed. The *parse* step includes removing stopwords, stemming and switching from numerical values to special digit tokens. For instance, if the utterance is *Is the cd or dvd included?*, then the parsed query would be like [*'is'*, *'cd'*, *'or'*, *'dvd'*, *'includ'*]. The tokens will be further vectorized into an array of numbers.

The closest matching embedding will then be found by comparing the vector to each vectorized question in the knowledge base. As a result, the user will receive the response that corresponds to this closest embedding. The closest response to the updated query is given back straight in the Direct-to-Answer scenario, without being matched to the question.

B. Data Preprocessing

We performed several preprocessing steps on the dataset before using it. Following the preparation of the data, we statistically analyzed the corpus. In user utterances, the average length of a question is 14.1 tokens, the average length of a response is 31.9 tokens, and 40.8% of queries have a number in them. We have a total of 14,460,956 tokens in our vocabulary—42,389 unique tokens at the beginning, before delexicalization and filtering, and 19,246 unique tokens at the

end. The data preprocessing steps are carried out through three main phases:

- **Cleaning** consists of eliminating unnecessary textual elements, like capitalization and stopword elimination. To aid in tokenization, we also had to substitute punctuation on a case-by-case basis. This is significant because electronic products often include references to items that include versions such as *'Kindle v2.0'* which include punctuation, but should not be split.
- **Normalization** entails mapping concepts inside the scheme or lemmatizing and stemming words to reduce language. For these reasons, we choose to use the default NLTK stemming package. Also, we swap out any incredibly uncommon words with the *<unk>* token. For example, *fujifilm xm* in "Would the lens fit on a *fujifilm xm*?" would be considered as rare.
- **Delexicalization** consists of substituting numbers to their underlying forms represented by the number of digits. So for example, 640 would be replaced with *<3-digit>* and 20 would be replaced with *<2-digit>*. This step essentially replaced nearly 10,000 unique number tokens with just 9 special tokens, as a result of the vocabulary's exponential growth.

C. Algorithms

We applied four major algorithms including BM25, Word2Vec, GloVe and FastText, which are introduced as below. All four types are effectively sentence embeddings which can map any given query or answer into the same vector space for comparison.

- **BM25** is the state-of-art TF-IDF-like algorithm for information retrieval. The inverse-document-frequency part is derived from a binary independence model, and the term-frequency part is a saturation function that approximates a complicated Poisson distribution model that takes into account the document length [17]. Given a query Q , which contains keywords $q_1, q_2, \dots, q_n$, the BM25 score for certain document can be calculated as below:

$$\text{score}(D, Q) = \sum_{i=1}^n \text{IDF}(q_i) \cdot \frac{f(q_i, D) \cdot (k+1)}{f(q_i, D) + k \cdot (1-b+b \cdot \frac{|D|}{\text{avgdl}})} \quad (1)$$

where $f(q_i, D)$ is the term frequency of q_i in document D . $|D|$ is the length of document D , while avgdl is the average length of all documents. k and b are parameters and usually $k \in [1.2, 2.0]$ and $b = 0.75$. The $\text{IDF}(q_i)$ is the IDF part (inverse-document-frequency), which is usually given as below:

$$\text{IDF}(q_i) = \log \frac{N - n(q_i) + 0.5}{n(q_i) + 0.5} \quad (2)$$

where N is the total number of documents, and $n(q_i)$ is the number of documents that contains q_i .

- **Word2Vec** is a model produces word embedding from corpus. It has two major implementations that are complements of each other. The first is the Continuous Bag Of

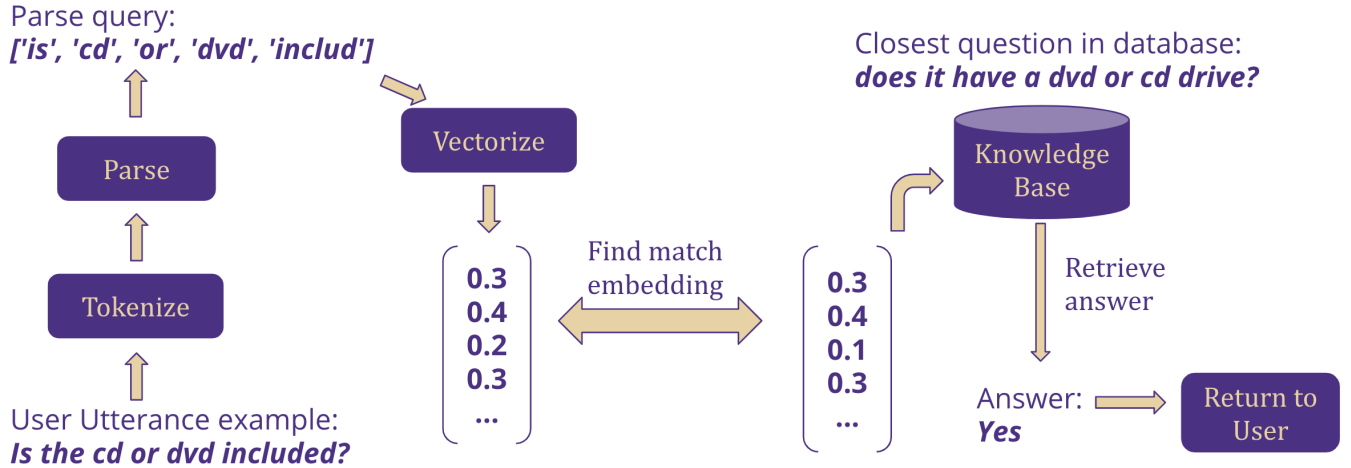


Fig. 2. Solution architecture of our chatbot.

TABLE I
ALGORITHM PERFORMANCE

Retrieval Style	Algorithms			
	BM25	GloVe	word2Vec	FastText
Raw Answers	0.94%	Out of Memory	Out of Memory	Out of Memory
	6 hours 38 min	20 hours 14 min	32 hours 24 min	25 hours 30 min
Filtered Answers	8.00%	2.25%	2.25%	3.83%
	57 minutes	2 hours 52 min	5 hours 41 min	3 hours 35 min
Raw Questions	87.85%	72.25%	70.67%	83.67%
	1.5 minutes	1.8 minutes	2.4 minutes	1.9 minutes

Words (CBOW) in which given surrounding words, the model aims to predict the missing word [17]. For example, given Today is a really --- day. the model would be expected to produce beautiful, nice or wonderful. It is normalized by:

$$V_{w_{I,c}}^{(new)} = V_{w_{I,c}}^{(old)} - \eta \cdot \frac{1}{C} \cdot EH$$

The second method is called the Skip-gram method. In this case, given the center word, learn to predict surrounding words. For example, given beautiful, the model might be expected to produce the tokens [a, really, day, .] for a context window of length 2. It is normalized by:

$$V_{w_{I,c}}^{(new)} = V_{w_{I,c}}^{(old)} - \eta \cdot \frac{1}{C} \cdot EH$$

The overall prediction is done with a typical softmax:

$$p(w_o|w_I) = \frac{\exp(v'_{w_o} \top v_{w_I})}{\sum_{w=1}^W \exp(v'_w \top v_{w_I})}$$

Then, the weights of the trained network become the embeddings to use in the future.

- **GLoVe** stands for a Global and Local Vectors. Its creation starts by making a large term-frequency matrix, similar to TF-IDF which is the global view. The local view, which is each token's window context, is taken into account when building this matrix. After that, rank reduction from

millions of dimensions to a more manageable number, like 100 or 300 dim, is accomplished via global matrix factorization. We employed the 300 dim pre-trained embeddings for our purposes [18].

In more detail, global matrix factorization aka latent semantic analysis (LSA) can be reduced via singular value decomposition (SVD).

$$\begin{aligned} M &= U \Sigma V^* \\ U U^* &= I_m \\ V V^* &= I_n \end{aligned}$$

Since GLoVe looks at local context like Word2Vec, and global context like BM25, it is theoretically a more powerful embedding model.

- **FastText** is an algorithm that dissects each token into its constituent subwords in addition to examining the token's embedding [19]. A token like school, for instance, would have the trigram breakdown "sch, cho, hoo, ool," each of which would have its own embedding and score. Specifically, the scoring mechanism works as:

$$score(w, c) = \sum_{g \in G_w} \mathbf{z}_g \top \mathbf{v}_c$$

Then, The final token is calculated by averaging the sum of the n-gram embeddings and the whole token embedding, school. The capacity to handle unfamiliar words is the main advantage of this method because it

is highly probable that new words will contain known subword embeddings. This is particularly pertinent to our project because, as was already noted, there are a lot of uncommon product names that are only seen once or never in training.

V. RESULTS

In this section, we present the evaluation and comparison of the four major algorithms—BM25, Word2Vec, GloVe, and FastText—for their effectiveness in generating sentence embeddings. The experiment performed on the datasets show that BM25 achieves the optimal performance in terms of both accuracy and time cost (Table I) compare to conventional models. On average the best performance have FastText and Word2Vec models (0.60). Comparing queries to raw questions, we can see that BM25 can achieve 87.85% accuracy, which is very high compared to GloVe and word2vec, and only slightly lower than FastText’s 83.67% accuracy. The time cost is also the lowest for BM25. We believe that this is due to the fact that the neural embedding matrix multiplications take up a lot of time.

For direct-to-answer approach using filtered answers, BM25 has the highest accuracy of 8.00% with the shortest time of 57min. However, this very low accuracy indicates that this approach is not suitable for this project. Therefore, we recommend the nearest query method. Finally, if you use raw answers, all approaches will have memory issues and you will not be able to complete the task. This is because you need to create a matrix of 1200x314x300 dimensions. The 314 comes from the size of the dataset, the 1200 is the number of examples and the 300 is the word dimension size.

VI. LIMITATIONS AND CONCLUSION

The major limitation is the lack of computing power. We come across out-of-memory issues when trying to compare a 310,000 x 300 dim matrix. Also in terms of time cost, it takes hours of computation required to loop through 10k+ examples. Thus we tried to mitigate the issue by pre-computing tokenization and then caching. Also, the results from the Direct-To-Answer approach show that the questions have very limited word overlap with answers. After all, intuitively, the words in the answer might not match the words in the question. For example, for the question-answer pair \What is the weather like?" and \Cloudy with a chance of rain", it's much easier to match with the revised queries like \What will the weather be today?" but not answers due to the unigram overlap from question to query.

In terms of algorithmic performance, comparing embeddings is still very fast compared to neural networks or Gradient-boosting algorithms. Also since we wrap the code into an independent class for each algorithm, we can easily transfer it into new embedding types for comparison. Also, our work is easy to interpret because we are just finding similar sentences. However, our algorithms do not have the processing power of neural networks to learn from data and can be quite

slow since all candidates must be processed during every query. In conclusion, we have learned a great deal about the trade-offs in word embeddings and would recommend taking a simple approach before diving into more complex neural methods.

REFERENCES

- [1] R. Egger, "Topic modelling: modelling hidden semantic structures in textual data," pp. 375–403, 2022.
- [2] A. Ritter, C. Cherry, and B. Dolan, "Data-driven response generation in social media," in *Empirical Methods in Natural Language Processing (EMNLP)*, 2011.
- [3] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," *arXiv preprint arXiv:1301.3781*, 2013.
- [4] H. Schütze, C. D. Manning, and P. Raghavan, *Introduction to information retrieval*, vol. 39. Cambridge University Press Cambridge, 2008.
- [5] D. Chen and C. D. Manning, "A fast and accurate dependency parser using neural networks," in *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, pp. 740–750, 2014.
- [6] H. L. Zongcheng Ji, Zhengdong Lu, "An information retrieval approach to short text conversation," in <https://arxiv.org/pdf/1408.6988.pdf>, 2014.
- [7] J. D. Williams, K. Asadi, and G. Zweig, "Hybrid code networks: practical and efficient end-to-end dialog control with supervised and reinforcement learning," *arXiv preprint arXiv:1702.03274*, 2017.
- [8] J. B. P. C. M. Z. L. J. Z. Zhao Yan, Nan Duan, "An information retrieval approach for chatbot engines using unstructured documents," 2016.
- [9] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, "Squad: 100,000+ questions for machine comprehension of text," *arXiv preprint arXiv:1606.05250*, 2016.
- [10] W. D. Alan Ritter, Colin Cherry, "Data-driven response generation in social media," in <https://www.aclweb.org/anthology/D11-1054>, 2011.
- [11] H. Li, *Learning to rank for information retrieval and natural language processing*. Springer Nature, 2022.
- [12] F. Zhu, W. Lei, C. Wang, J. Zheng, S. Poria, and T.-S. Chua, "Retrieving and reading: A comprehensive survey on open-domain question answering," *arXiv preprint arXiv:2101.00774*, 2021.
- [13] X. Li, P. Li, W. Bi, X. Liu, and W. Lam, "Relevance-promoting language model for short-text conversation," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, pp. 8253–8260, 2020.
- [14] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," *arXiv preprint arXiv:1301.3781*, 2013.
- [15] G. K. Vamsi, A. Rasool, and G. Hajela, "Chatbot: A deep neural network based human to machine conversation model," in *2020 11th International Conference on Computing, Communication and Networking Technologies (ICCCNT)*, pp. 1–7, IEEE, 2020.
- [16] H. K. Obayes, F. S. Al-Turaihi, and K. H. Alhussayni, "Sentiment classification of user's reviews on drugs based on global vectors for word representation and bidirectional long short-term memory recurrent neural network," vol. 23, pp. 345–353, 2021.
- [17] M. Kulkarni and S. Kale, "Information retrieval based improvising search using automatic query expansion," in *2021 Third International Conference on Intelligent Communication Technologies and Virtual Mobile Networks (ICICV)*, pp. 1226–1230, IEEE, 2021.
- [18] C. I. Eke, A. Norman, L. Shuib, F. B. Fatokun, and I. Omame, "The significance of global vectors representation in sarcasm analysis," in *2020 International Conference in Mathematics, Computer Engineering and Computer Science (ICMCECS)*, pp. 1–7, IEEE, 2020.
- [19] P. Bojanowski, E. Grave, A. Joulin, and T. Mikolov, "Enriching word vectors with subword information," *Transactions of the association for computational linguistics*, vol. 5, pp. 135–146, 2017.